

CMSC828G: Special Topics in Systems for Machine Learning

CUDA and Triton

Instructor: Yafan Huang



DEPARTMENT OF
COMPUTER SCIENCE

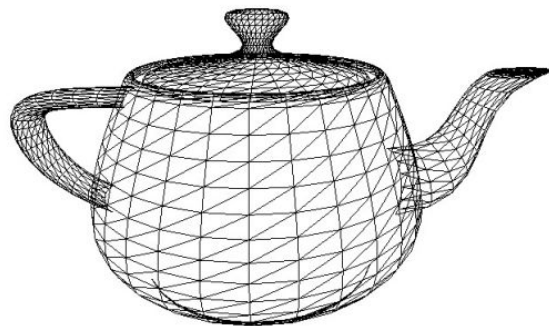
Before This Lecture

- Why still CUDA today?
 - Hardware changes. Principles persist.
 - AI makes systems intuition more valuable, not less.
 - Higher-level abstractions (DSL like Triton) still rest on GPU fundamentals.
 - For many scientific workloads, CUDA is still the portal to GPU.
 - And yes... people still hire kernel engineers. 😂

Outline

- Background
- CUDA Basics
- Optimizing CUDA Programs
- Triton Basics

What GPUs were Original Designed to do: 3D Rendering



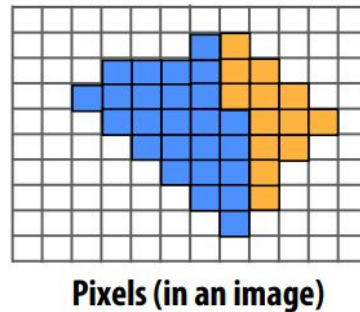
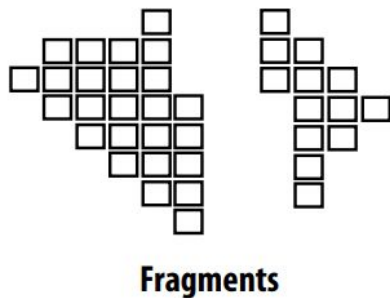
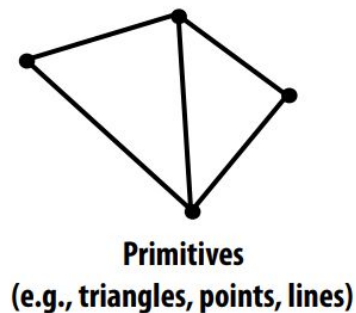
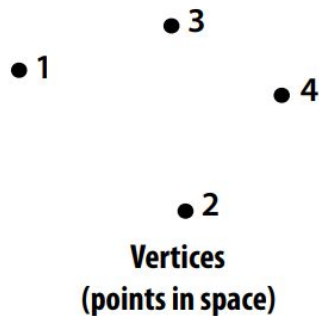
Input: description of a scene:
3D surface geometry

Output: Image of the scene

Simple definition of rendering task: computing how each triangle in 3D mesh contributes to appearance of each pixel in the image?

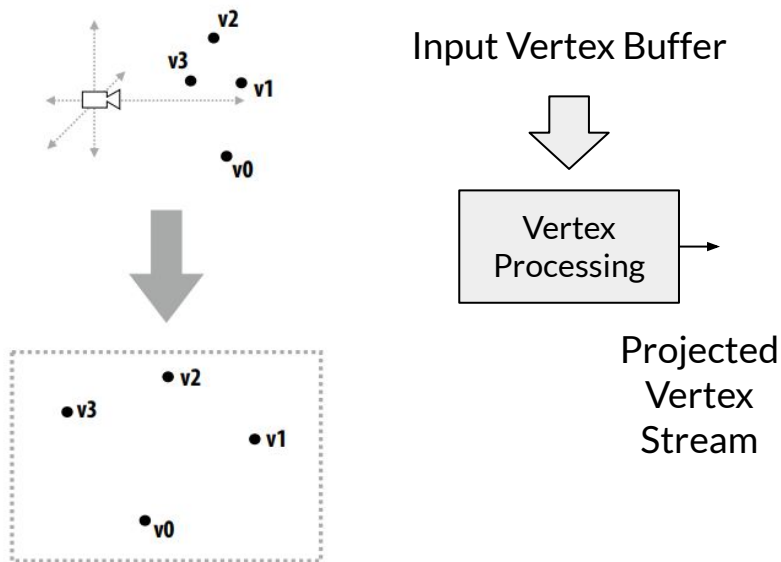
3D Rendering Task with GPU

- Real-time graphic primitives (entities)



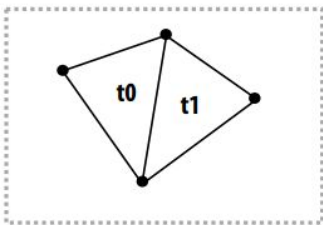
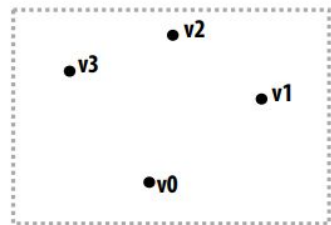
Rendering a Picture

- Step 1: given a scene position, compute where the vertices lie on screen

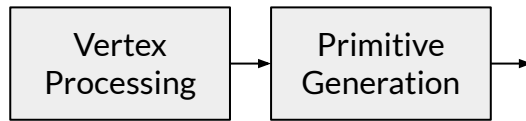


Rendering a Picture

- Step 2: group vertices into primitives



Input Vertex Buffer

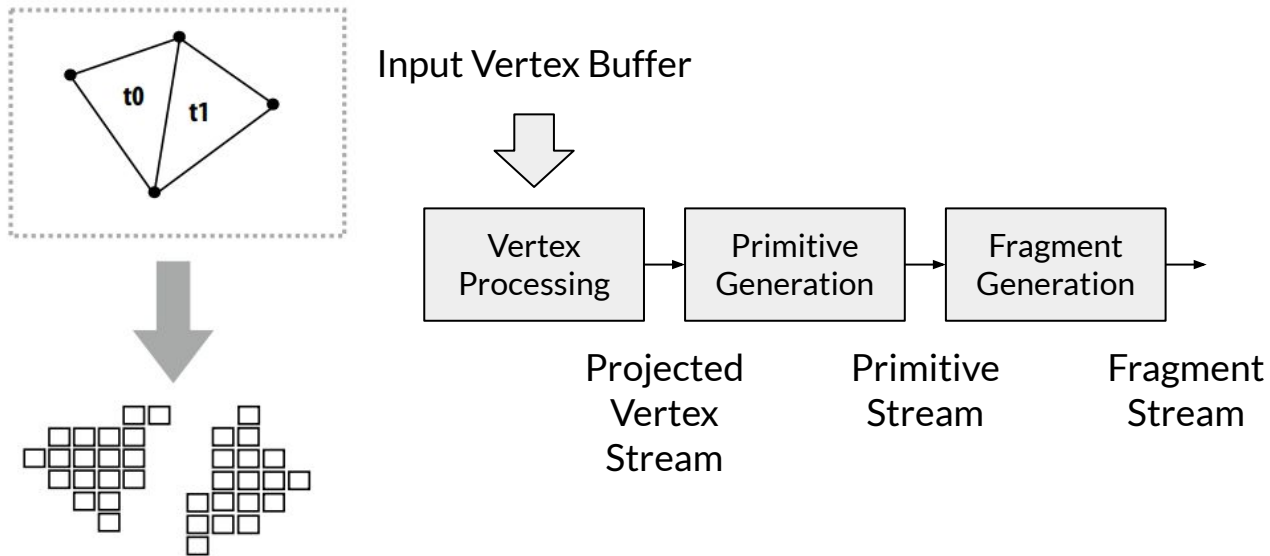


Projected
Vertex
Stream

Primitive
Stream

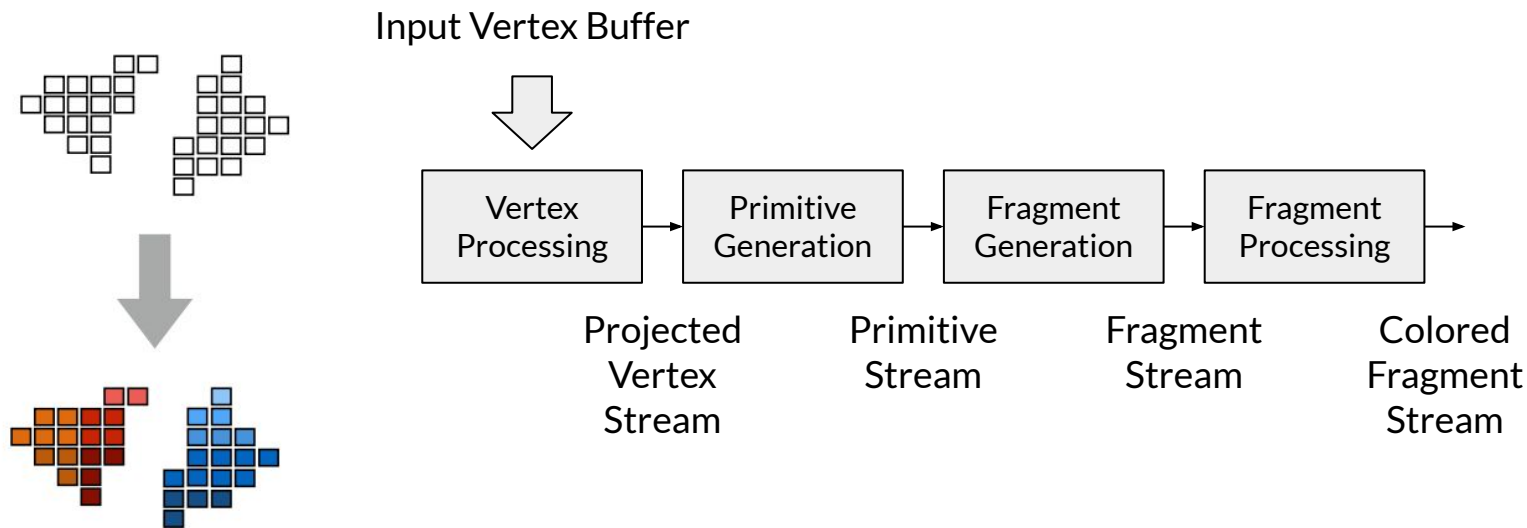
Rendering a Picture

- Step 3: generate one fragment for each pixel based on the primitive



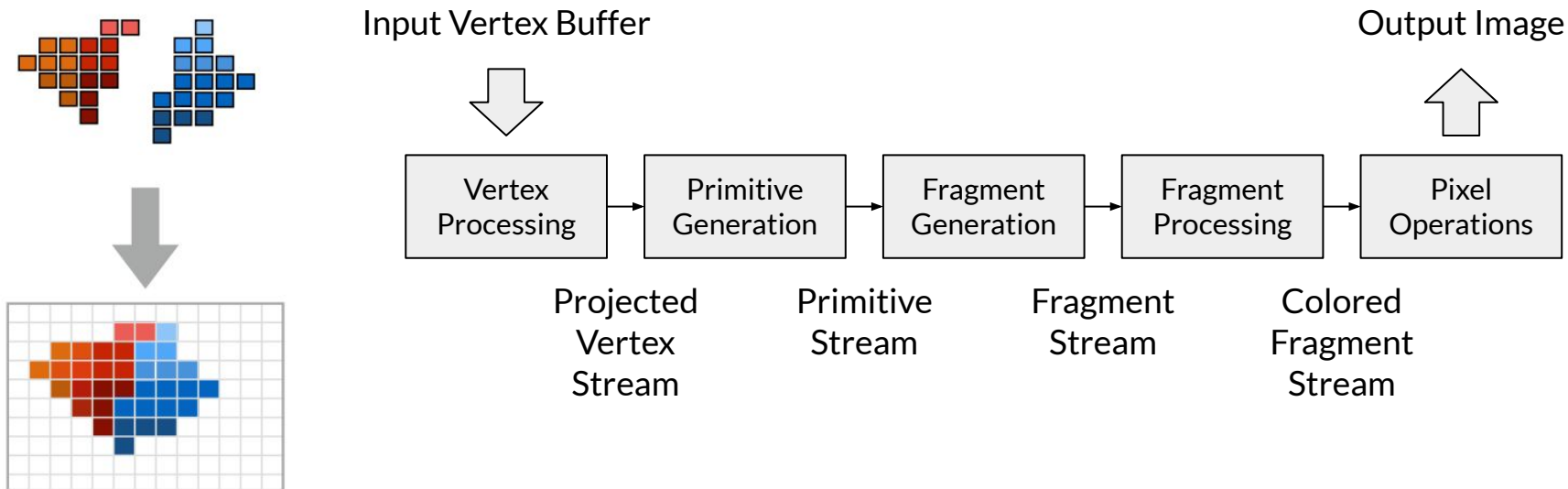
Rendering a Picture

- Step 4: compute color of primitives for each fragment (based on scene lighting, material properties, entity interactions, etc)



Rendering a Picture

- Step 5: post-processing for each pixel and generate output image



GPGPU: General Purpose Computations on GPU

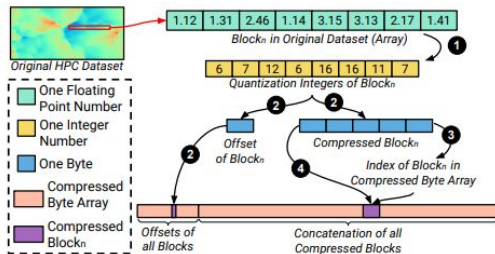
- GPGPU, short for General-purpose computing on graphics processing units, refers to the use of GPUs to perform computations beyond their traditional role of rendering graphics.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

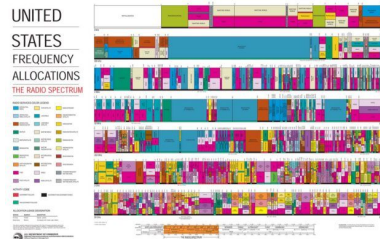
$$\begin{aligned} 1 \times 6 + 2 \times 8 &= 22 \\ 1 \times 5 + 2 \times 7 &= 19 \\ 3 \times 5 + 4 \times 7 &= 43 \\ 3 \times 6 + 4 \times 8 &= 50 \end{aligned}$$

8 multiplications

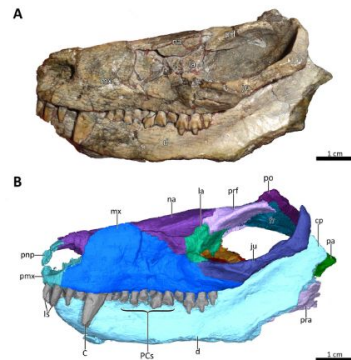
Matrix-Matrix
Multiplication



Data Reduction



Audio Processing

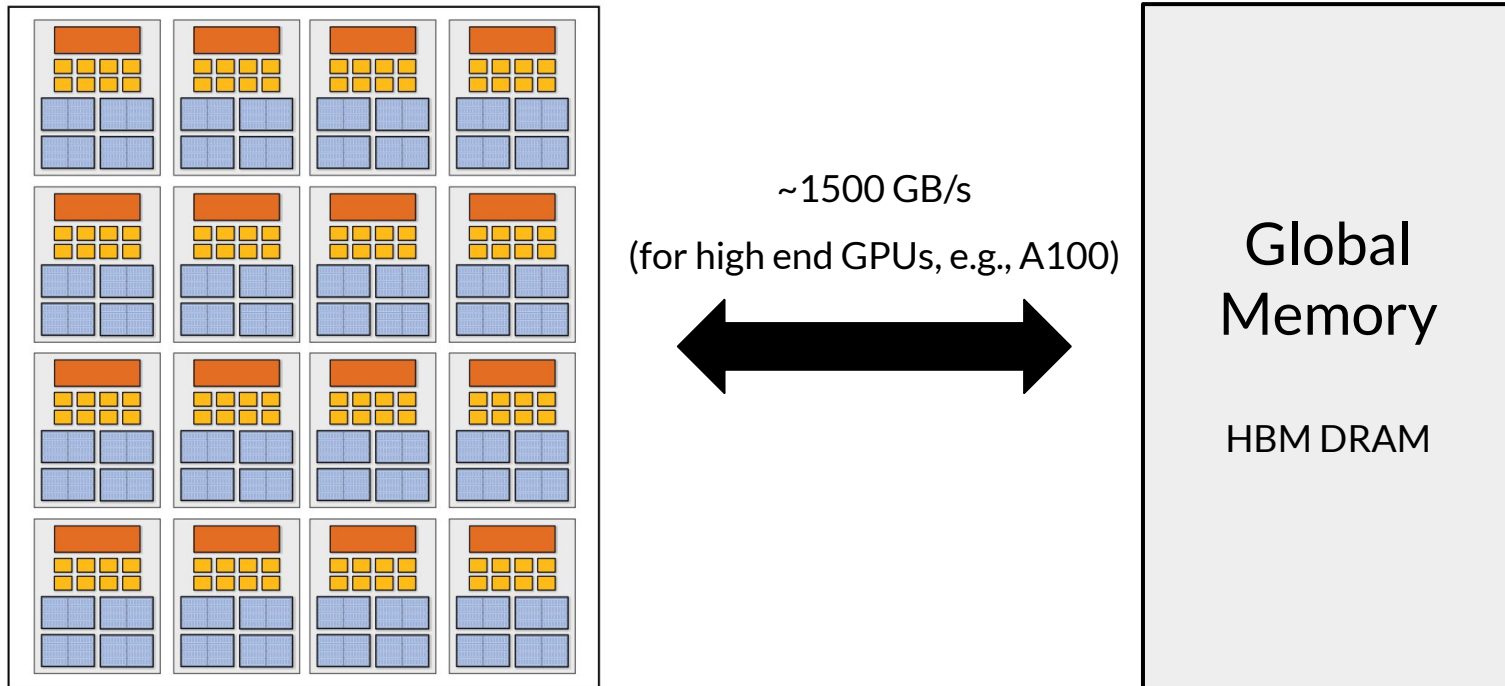


CT (Computed
Tomography) Scan

CUDA Programming Language

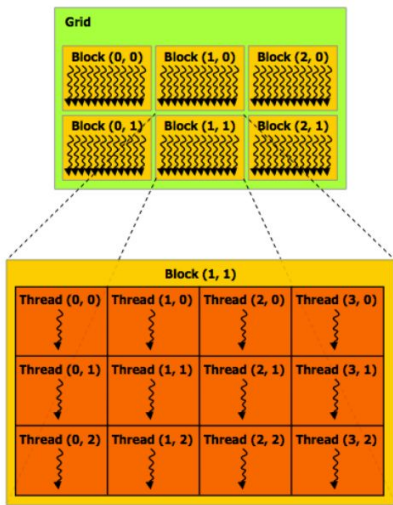
- Introduced in 2007 with NVIDIA Tesla architecture.
- “C-like” language to express programs that run on GPUs using compute-mode hardware interface.
- Relatively low-level: CUDA’s abstractions closely match the capacities/performance characteristics of modern GPUs.
- Note: OpenCL is an open standards version of CUDA
 - CUDA only runs on NVIDIA GPUs.
 - OpenCL runs on CPUs and GPUs from many vendors.
 - Poor documentations, unfriendly developing environments, poor performance, etc. CUDA is much more popular now.

Basic GPU Architecture



CUDA Program Consists of Hierachy of Concurrent Threads

- Thread IDs can be up to 3-dimensional (2D example below)
- Multi-dimensional thread IDs are convenient for problems that are naturally N-D



```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3, 1);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/threadsPerBlock.y, 1);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will trigger execution of 72 CUDA threads:
// 6 thread blocks of 12 threads each
matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

Regular application thread running on CPU (the “host”)

Basic CUDA Syntax

Host code: serial execution
Running as part of normal C/C++
Application on CPU

```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3, 1);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/threadsPerBlock.y, 1);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will trigger execution of 72 CUDA threads:
// 6 thread blocks of 12 threads each
matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

Regular application thread running on CPU (the “host”)

CUDA device code: kernel function
(__global__ denotes a CUDA kernel
function) runs on GPU

Function call returns when all
threads have terminated

```
// kernel definition

__global__ void matrixAdd(
    float A[Ny][Nx], float B[Ny][Nx], float C[Ny][Nx])
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    C[j][i] = A[j][i] + B[j][i];
}
```

CUDA kernel function

CUDA Execution Model

Host
(serial execution)



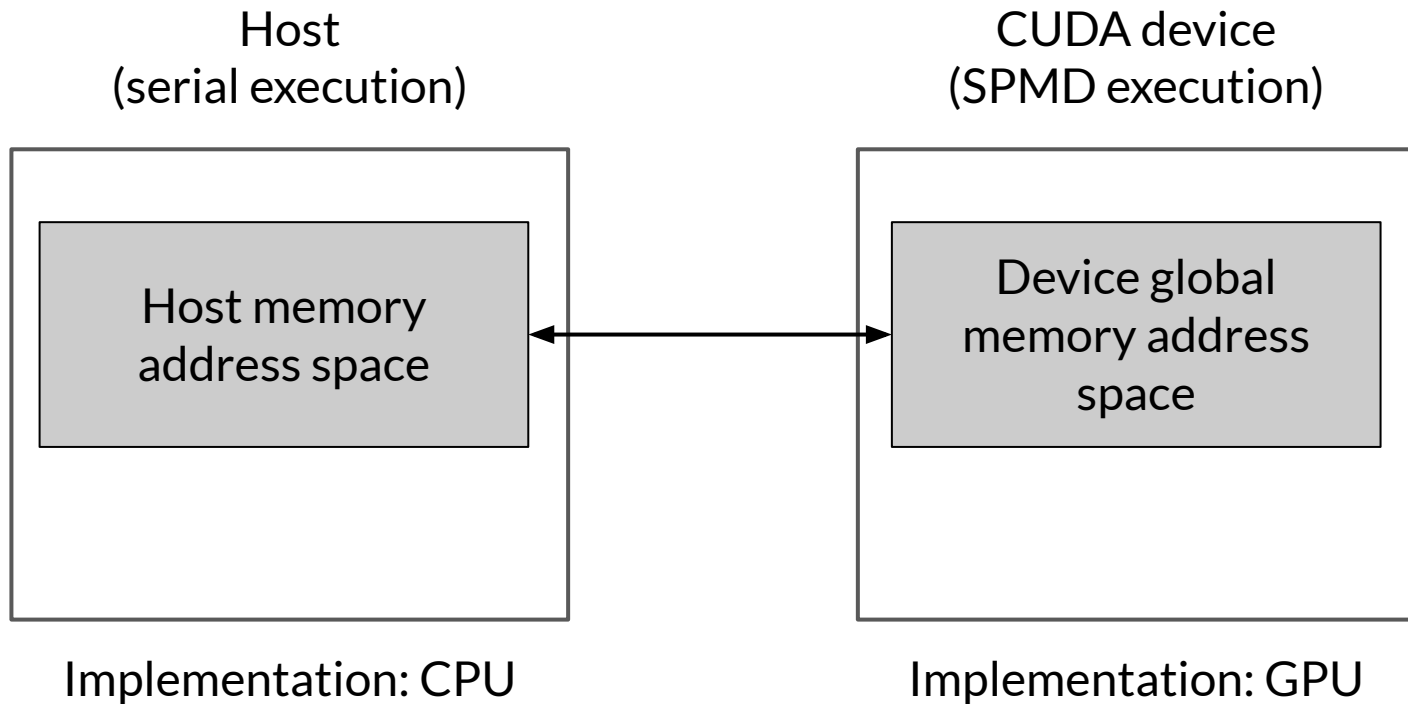
Implementation: CPU

CUDA device
(SPMD execution)



Implementation: GPU

CUDA Execution Model



Memcpy Primitive

```
float* A = new float[N]; // allocate buffer in host mem

// populate host address space pointer A
for (int i=0; i<N; i++)
    A[i] = (float)i;

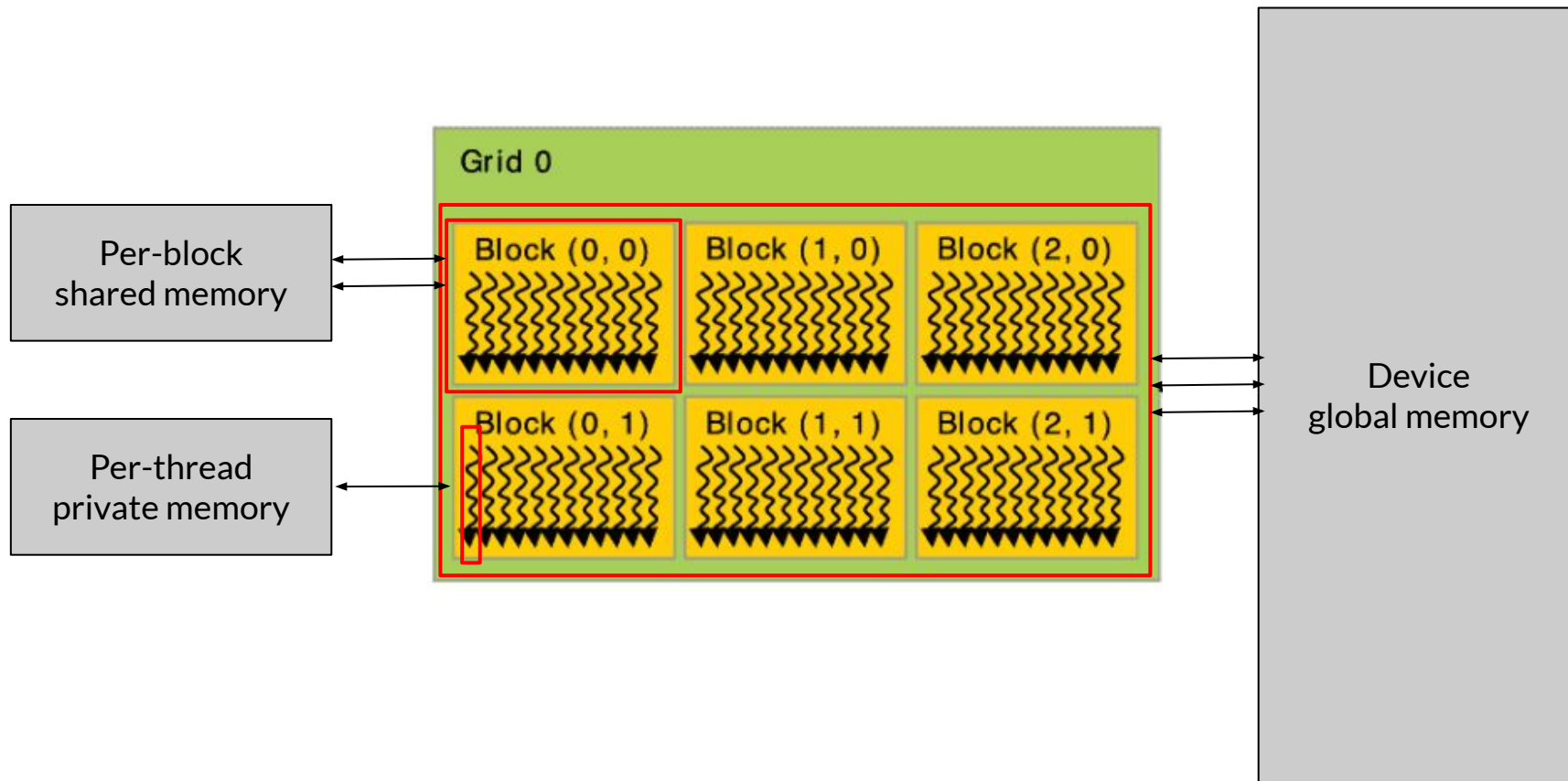
int bytes = sizeof(float) * N
float* deviceA; // allocate buffer in
cudaMalloc(&deviceA, bytes); // device address space

// populate deviceA
cudaMemcpy(deviceA, A, bytes, cudaMemcpyHostToDevice);

// note: deviceA[i] is an invalid operation here (cannot
// manipulate contents of deviceA directly from host.
// Only from device code.)
```

After kernel finishes execution, copy data back to CPU by
`cudaMemcpy(new_A, deviceA, bytes, cudaMemcpyDeviceToHost)`

CUDA Device Memory Model



CUDA Synchronization Constructs

- `__syncthreads()`
 - Barrier: wait for all threads in the block to arrive at this point
- Atomic operations
 - e.g., `float atomic add(float* addr, float amount)`
 - Atomic operations on both global memory and shared memory var
- Host/device synchronization
 - Implicit barrier across all threads at return of kernel

Acknowledgement

- Parts of these slides were adapted from and inspired by materials from CMU 15-418/15-618: Parallel Computer Architecture and Programming, particularly the [lecture on GPU architecture](#). I gratefully acknowledge the course instructors and authors of these materials.